HLD Level 1.4

Caching - C++ / Systems Interview Notes

> **Goal: understand why caches improve latency and reduce backend load, where to place them, how data enters/leaves them, and how to reason about staleness, eviction and failure.**

1. What is a cache?

A cache stores a copy of frequently or recently needed data in a faster-access location so repeated work can avoid a slower database, disk, network call or computation.

```
Client -> Service -> Cache -> HIT -> return
|
+-> MISS -> Database
```

The cache is usually an optimization layer, not automatically the source of truth.

2. Why cache?

Typical goals are lower latency, reduced database/storage load, reduced network calls, reduced repeated computation, and increased read throughput.

> **Interview rule: never say only 'I'll add a cache.' State what you cache, where, why, how it is invalidated, and what happens on a miss/failure.**

3. Cache hit and miss

Hit: requested value exists and is usable in cache.
Miss: value is absent/expired, so the system retrieves or computes it from the underlying source.

```
Hit ratio = cache hits / total cache lookups
```

A cache with a poor hit ratio may add complexity without providing enough benefit.

4. Local vs distributed cache

Local/in-process cache: stored inside each application instance. Extremely fast and avoids network access, but each instance has its own copy and consistency across replicas becomes harder.

Distributed cache: separate shared cache service accessed by multiple application instances. Easier to share state but adds a network hop and its own scaling/failure concerns.

```
Local:
Service A [cache A] Service B [cache B]

Distributed:
Service A ---\
Shared Cache
Service B ---/
```

For C++, a local cache may be an in-memory map/LRU structure protected for concurrency. Distributed caching moves much of that state outside the process.

5. What should be cached?

Good candidates are frequently read data, expensive database queries, expensive computations, relatively stable metadata, or data where some staleness is acceptable.

Poor candidates include highly volatile values requiring strict freshness, rarely reused values, sensitive data without appropriate controls, and data whose invalidation cost outweighs the benefit.

6. Cache-aside / lazy loading

One of the most common patterns: application checks cache first. On miss it reads the database, puts the result in cache, then returns it.

```
read(key):
value = cache.get(key)
if hit: return value
value = DB.get(key)
cache.put(key,value)
return value
```

Advantages: only requested data is cached; application controls behavior.
Trade-offs: first request is slower; cache and DB can diverge; application handles cache logic.

7. Read-through cache

The application asks the cache for data; on a miss, the caching layer is responsible for retrieving/loading the underlying value. This hides some loading logic from application code.

```
Application -> Cache -> on miss -> Data Source
```

8. Write-through

A write is synchronously applied through the cache to the backing store before success is returned.

```
Application -> Cache -> Database -> acknowledge
```

This can keep cache/backing data more synchronized but adds write-path latency.

9. Write-back / write-behind

The application writes to the cache and the backing store is updated asynchronously later.

```
Application -> Cache -> ACK
|
+--> async DB write
```

This can provide fast writes and batching opportunities, but durability and failure handling become much harder. If dirty cached data is lost before persistence, data may be lost.

> **Do not casually choose write-back for critical financial/order data unless you can explain durability and recovery.**

10. TTL - Time To Live

A TTL causes cached entries to expire after a period. TTL bounds how long stale data may survive and allows old entries to disappear automatically.

```
put(user:42, value, TTL=60s)

t=0 -> valid
t=30 -> valid
t=61 -> expired
```

TTL selection is a trade-off: short TTL improves freshness but increases misses/backend load; long TTL improves hit rate but increases potential staleness.

11. Cache invalidation

When the source of truth changes, cached copies may become stale. Common approaches include explicit invalidation/update on writes, TTL-based expiration, versioning, or event-driven invalidation.

> **Caching is easy. Keeping cached data sufficiently correct is the hard part.**

12. Update vs invalidate

After DB update you can either update the cached value or delete/invalidate it. Invalidating is often simpler: the next read repopulates from the source of truth. Updating avoids a miss but creates more synchronization logic.

```
Option A: DB write -> cache.delete(key)
Option B: DB write -> cache.put(key,newValue)
```

13. Eviction policies

Memory is finite, so a cache eventually needs to remove entries.

LRU: evict least recently used. Good when recently accessed items are likely to be reused.

LFU: evict least frequently used. Useful when long-term popularity matters.

FIFO: evict oldest inserted entries; simple but does not reflect access patterns.

TTL expiration: removes expired values, but is not always sufficient as the only capacity policy.

Your C++ LRU-cache DSA knowledge connects directly here: unordered_map + doubly linked list can provide O(1) lookup and recency updates.

14. Cache stampede / thundering herd

Suppose a very popular key expires. Thousands of requests simultaneously miss and all query the database.

```
Popular key expires
|
10000 requests -> MISS MISS MISS
| | | | |
+-------> Database overload
```

Mitigations include per-key locking/request coalescing, refreshing before expiration, staggered/randomized TTLs, stale-while-revalidate, and protecting the backend with limits.

15. TTL jitter

If millions of entries are inserted with the exact same TTL, they may expire together and create a load spike. Adding small random variation to expiration times spreads the refresh work.

```
Instead of all TTL = 300s
use approximately 270..330s where appropriate
```

16. Cache penetration

Repeated requests for keys that do not exist can repeatedly miss the cache and hit the database. One mitigation is **negative caching**: temporarily cache the fact that a key is absent. Probabilistic filters can also help in some architectures.

17. Cache failure

If cache is only an optimization and it fails, traffic may fall through to the database. But that can suddenly multiply database load and cause a cascading failure.

```
Cache healthy: 90% hit -> DB sees 10% traffic
Cache down: DB may see ~100% traffic
```

Design cache failure behavior. 'Just go to the database' may overload the database.

18. Cold cache / warm-up

After restart or deployment, an empty cache can cause a burst of misses. Strategies include lazy warm-up, preloading known hot keys, gradual traffic shifting, or retaining an external distributed cache across app restarts.

19. Consistency and stale data

Caching introduces another copy of data. Ask: how stale may this copy be? A product description might tolerate seconds/minutes of staleness; an authoritative bank balance may not.

The freshness requirement determines TTL and invalidation strategy.

20. Caching and multiple service instances

```
-> Service A [local cache]
Client -> LB
-> Service B [local cache]
```

If A and B maintain independent local caches, one may hold old data while the other has new data. Solutions may include short TTLs, invalidation events, version checks, or a shared distributed cache depending on requirements.

21. C++ local-cache concerns

For an in-process C++ cache, HLD can lead to LLD/system questions: unordered_map/map choice, LRU list, mutex/shared_mutex, sharding locks, contention, memory ownership, smart pointers, object lifetime, memory limits, eviction cost and allocator behavior.

```
Request threads
|
+--> Cache shards
[lock + map + LRU]
[lock + map + LRU]
[lock + map + LRU]
```

One global mutex may become a bottleneck under high concurrency. Sharded caches or carefully designed concurrency strategies can reduce contention.

22. Cache memory estimation

Do not estimate only payload bytes. Each cached item can also consume key memory, hash-table/node overhead, pointers, allocator metadata and alignment.

```
1,000,000 entries x 1 KB payload ~= 1 GB payload alone
actual process memory > 1 GB
```

23. Cache vs database

A cache normally should not be introduced merely because a database exists. First identify the bottleneck/read pattern. Databases already use internal caching and indexes; an application cache adds operational and consistency complexity.

Use a cache when repeated access or expensive work justifies the extra copy and its consistency cost.

24. Example - device configuration

Suppose 1 million devices repeatedly request configuration, but configuration changes rarely. This is a strong cache candidate.

```
Device -> Config Service -> Cache
| MISS
v
Database
```

On configuration update, write the database and invalidate/update the cached configuration. Choose TTL based on how quickly devices must observe changes.

25. Example - live stock/order state

Be cautious caching authoritative rapidly changing transactional state. Market-data display caches can be useful, but order execution, balances and authoritative positions have stronger correctness requirements. Always distinguish display/derived data from the source of truth.

26. Interview decision checklist

1. **What exact data/work am I caching?**
2. **Why is it expensive without a cache?**
3. **Local or distributed?**
4. **What is the source of truth?**
5. **How stale may data be?**
6. **TTL?**
7. **How are writes handled?**
8. **Update or invalidate?**
9. **Eviction policy?**
10. **What happens on cache miss/failure?**
11. **Can a hot-key stampede occur?**
12. **What is the memory budget?**

27. Common mistakes

'Use Redis' without reasoning: technology before requirements.

Cache everything: wastes memory and creates consistency problems.

No TTL/invalidation: stale values can live indefinitely.

Cache is always available: false.

Fallback DB can handle all traffic: not necessarily.

TTL solves all invalidation: it only bounds staleness.

Ignoring hot keys: one key can dominate traffic.

Payload size = cache memory: ignores data-structure overhead.

28. Practice scenarios

1. Product details are read millions of times but change once a day. Would you cache them? What strategy?

2. A popular cached key expires and 20,000 requests arrive simultaneously. What happens and how would you prevent it?

3. Your cache serves 95% of reads and suddenly fails. What happens to the database?

4. Two C++ service instances use independent local caches. A value changes in the DB. What consistency issue appears?

5. When would you prefer LRU over LFU?

6. A bank balance changes frequently and correctness is critical. Would you casually cache it with a 5-minute TTL? Why not?

29. 60-second revision

Cache: faster copy used to avoid slower/repeated work.
Cache-aside: app checks cache, loads DB on miss.
Write-through: synchronously persist through cache.
Write-back: persist asynchronously; higher durability complexity.
TTL: bounds lifetime/staleness.
LRU: evict least recently used.
LFU: evict least frequently used.
Stampede: many simultaneous misses for a hot key.
Local cache: fastest but per-instance consistency issues.
Distributed cache: shared but network/failure overhead.
Always ask: source of truth, freshness, invalidation and failure behavior.

30. Ready for Level 1.5 when...

You can justify what to cache, choose local vs distributed, explain cache-aside/write-through/write-back, select TTL/eviction approaches, and reason about invalidation, stampedes, cold caches and cache failure.